PSTAT 126 Final Project

Conrad Fritz

2026-06-06

Part 1: Data Description and Descriptive Statistics:

1. Select a random sample.

```
n <- nrow(data)/2
data.samp <- slice_sample(data, prop = 0.2)
summary(data.samp)
```

```
##        X             carat            cut                color          
##  Min.   :    6   Min.   :0.2000   Length:10788       Length:10788      
##  1st Qu.:13061   1st Qu.:0.4000   Class :character   Class :character  
##  Median :26595   Median :0.7100   Mode  :character   Mode  :character  
##  Mean   :26752   Mean   :0.8002                                        
##  3rd Qu.:40410   3rd Qu.:1.0400                                        
##  Max.   :53940   Max.   :4.0100                                        
##    clarity              depth           table          price       
##  Length:10788       Min.   :50.80   Min.   :44.0   Min.   :  336  
##  Class :character   1st Qu.:61.00   1st Qu.:56.0   1st Qu.:  961  
##  Mode  :character   Median :61.80   Median :57.0   Median : 2453  
##                     Mean   :61.75   Mean   :57.5   Mean   : 3936  
##                     3rd Qu.:62.50   3rd Qu.:59.0   3rd Qu.: 5304  
##                     Max.   :79.00   Max.   :76.0   Max.   :18823  
##        x                y               z          
##  Min.   : 3.73   Min.   : 3.680   Min.   :0.000  
##  1st Qu.: 4.72   1st Qu.: 4.740   1st Qu.:2.920  
##  Median : 5.71   Median : 5.720   Median :3.530  
##  Mean   : 5.74   Mean   : 5.742   Mean   :3.543  
##  3rd Qu.: 6.55   3rd Qu.: 6.540   3rd Qu.:4.030  
##  Max.   :10.14   Max.   :10.100   Max.   :6.270
```

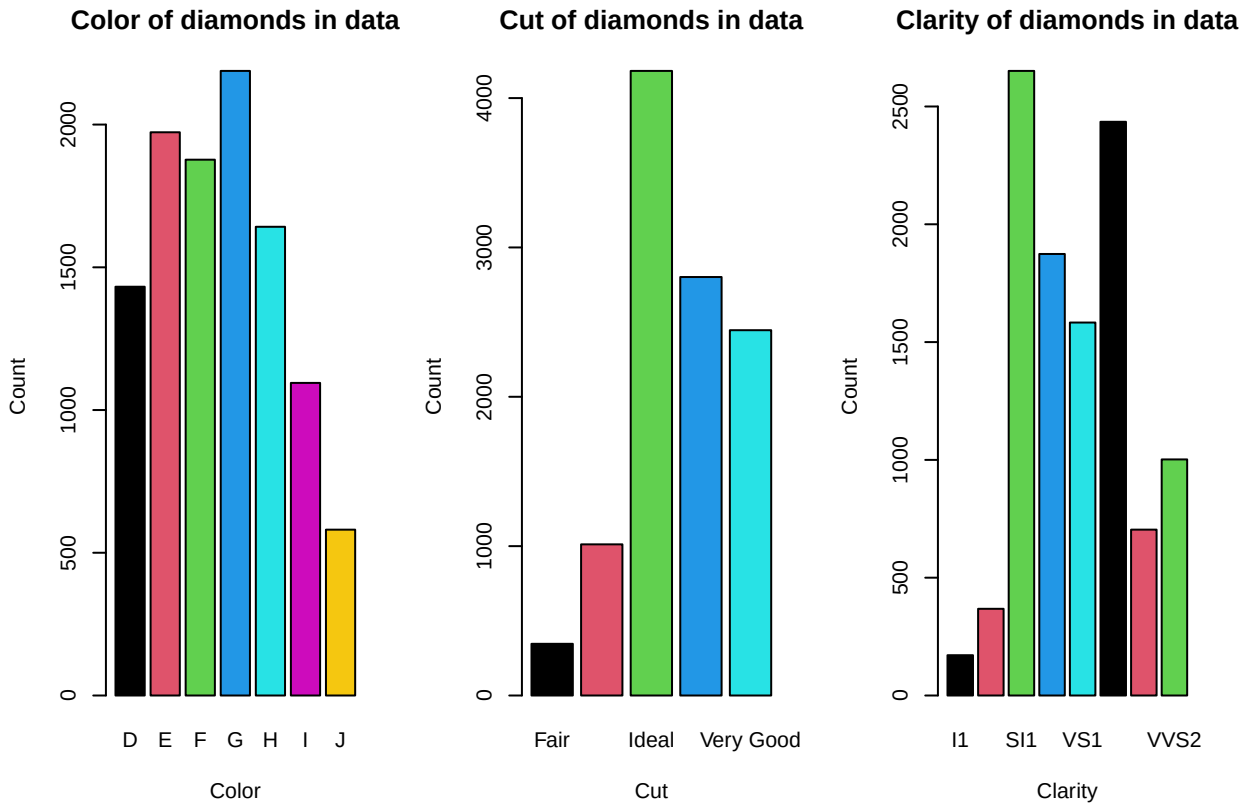
2. Describe all variables.

```
summary(data)
```

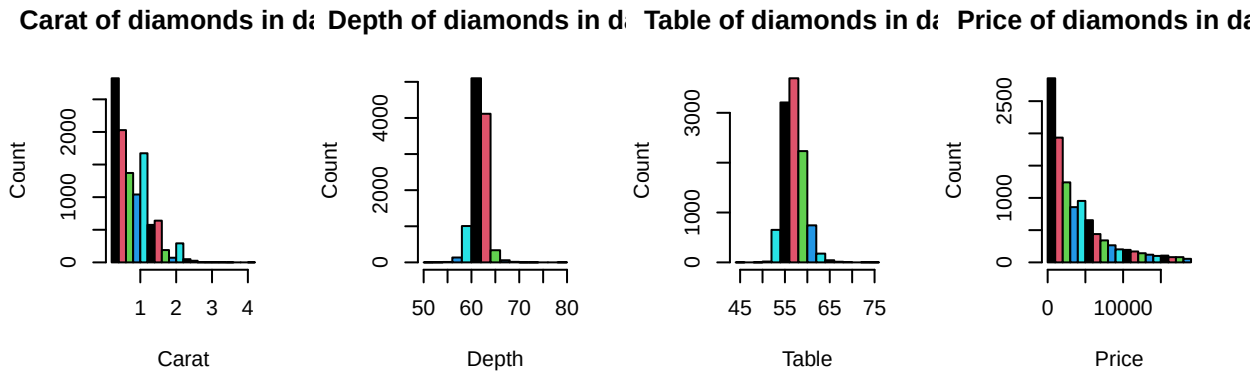
```
##        X             carat            cut                color          
##  Min.   :    1   Min.   :0.2000   Length:53943       Length:53943      
##  1st Qu.:13486   1st Qu.:0.4000   Class :character   Class :character  
##  Median :26972   Median :0.7000   Mode  :character   Mode  :character  
##  Mean   :26972   Mean   :0.7979
```

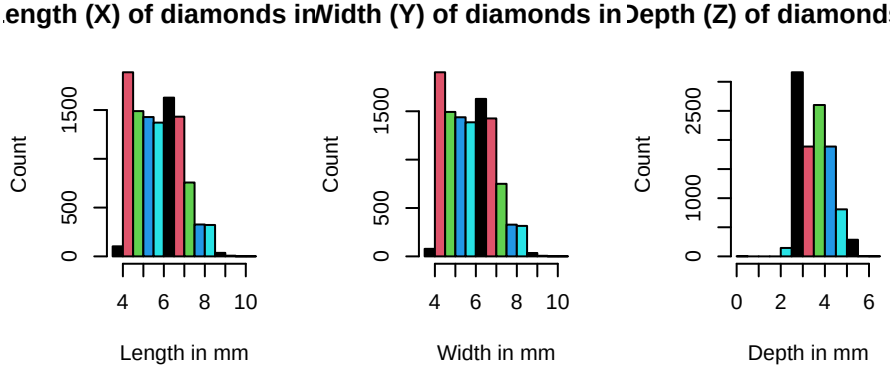
```
##  3rd Qu.:40458   3rd Qu.:1.0400
##  Max.   :53943   Max.   :5.0100
##    clarity              depth           table           price
##  Length:53943       Min.   :43.00   Min.   :43.00   Min.   :  326
##  Class :character   1st Qu.:61.00   1st Qu.:56.00   1st Qu.:  950
##  Mode  :character   Median :61.80   Median :57.00   Median : 2401
##                     Mean   :61.75   Mean   :57.46   Mean   : 3933
##                     3rd Qu.:62.50   3rd Qu.:59.00   3rd Qu.: 5324
##                     Max.   :79.00   Max.   :95.00   Max.   :18823
##        x                y               z
##  Min.   : 0.000   Min.   : 0.000   Min.   : 0.000
##  1st Qu.: 4.710   1st Qu.: 4.720   1st Qu.: 2.910
##  Median : 5.700   Median : 5.710   Median : 3.530
##  Mean   : 5.731   Mean   : 5.735   Mean   : 3.539
##  3rd Qu.: 6.540   3rd Qu.: 6.540   3rd Qu.: 4.040
##  Max.   :10.740   Max.   :58.900   Max.   :31.800
```

```r
# Categorical Data
par(mfrow = c(1, 3))
table(data.samp$color) %>%
    barplot(main = "Color of diamonds in data", ylab = "Count",
        xlab = "Color", col = 1:7)
table(data.samp$cut) %>%
    barplot(main = "Cut of diamonds in data", ylab = "Count",
        xlab = "Cut", col = 1:5)
table(data.samp$clarity) %>%
    barplot(main = "Clarity of diamonds in data", ylab = "Count",
        xlab = "Clarity", col = 1:5)
```




```
par(mfrow = c(2, 4))
data.samp$carat %>%
    hist(main = "Carat of diamonds in data", ylab = "Count",
        xlab = "Carat", col = 1:5)
data.samp$depth %>%
    hist(main = "Depth of diamonds in data", ylab = "Count",
        xlab = "Depth", col = 1:5)
data.samp$table %>%
    hist(main = "Table of diamonds in data", ylab = "Count",
        xlab = "Table", col = 1:5)
data.samp$price %>%
    hist(main = "Price of diamonds in data", ylab = "Count",
        xlab = "Price", col = 1:5)
data.samp$x %>%
    hist(main = "Length (X) of diamonds in data", ylab = "Count",
        xlab = "Length in mm", col = 1:5)
data.samp$y %>%
    hist(main = "Width (Y) of diamonds in data", ylab = "Count",
        xlab = "Width in mm", col = 1:5)
data.samp$z %>%
    hist(main = "Depth (Z) of diamonds in data", ylab = "Count",
        xlab = "Depth in mm", col = 1:5)
```





There are 7 colors (D-J), 5 Types of Cuts, 8 Clarity categories, a range from 0 to 4 carats, a range of 45-75 mm (?) depth, Prices ranging from 0 to over 15k, the length of a diamond from 0 to 10 mm, the width of a diamond from 0-30 mm, and the depth of a diamond from 0-6 mm.

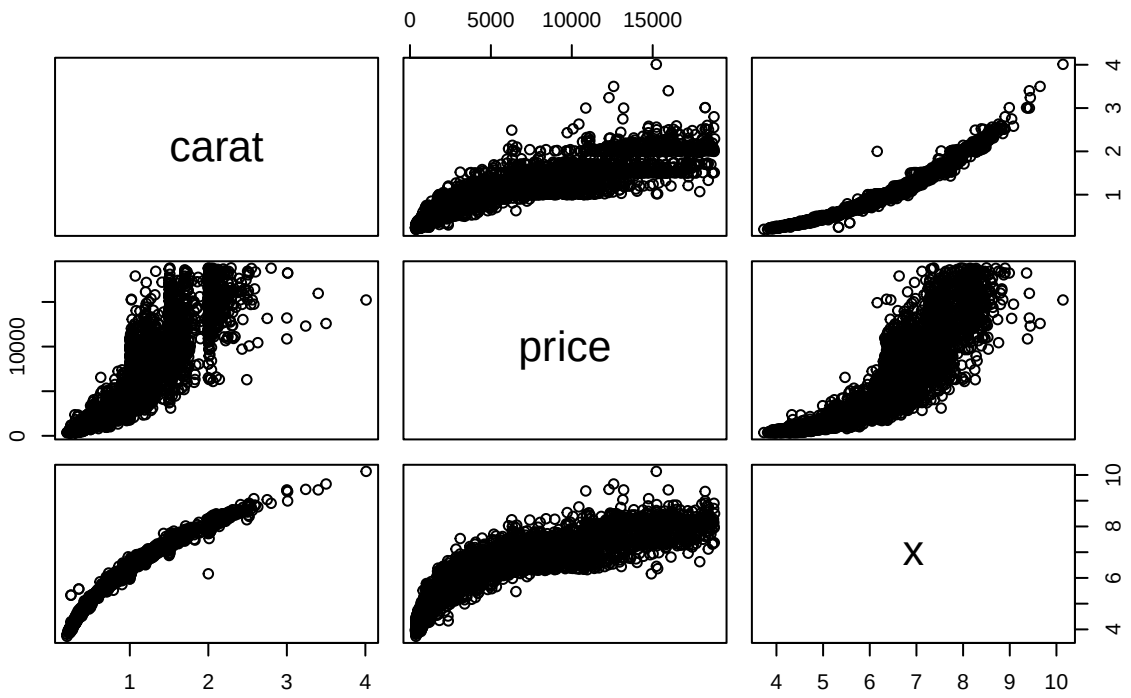
3. For this problem, we're going to pick carat, price and the length (x) in mm of the diamond.

```
vars <- data.samp %>%
    select(carat, price, x)
matrix <- cor(vars)

matrix
```

```
##           carat     price         x
## carat 1.0000000 0.9191466 0.9776310
## price 0.9191466 1.0000000 0.8844389
## x     0.9776310 0.8844389 1.0000000
```

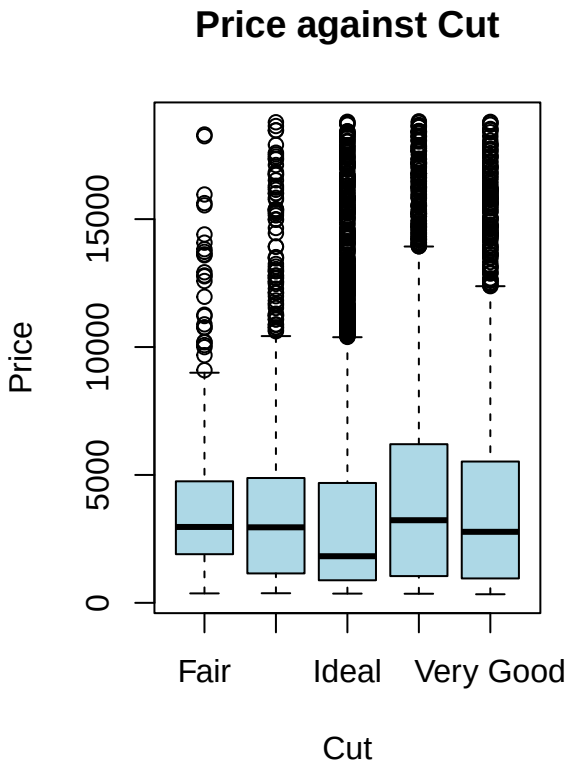
```
plot(vars, main = "Scatterplot Matrix of Variables")
```

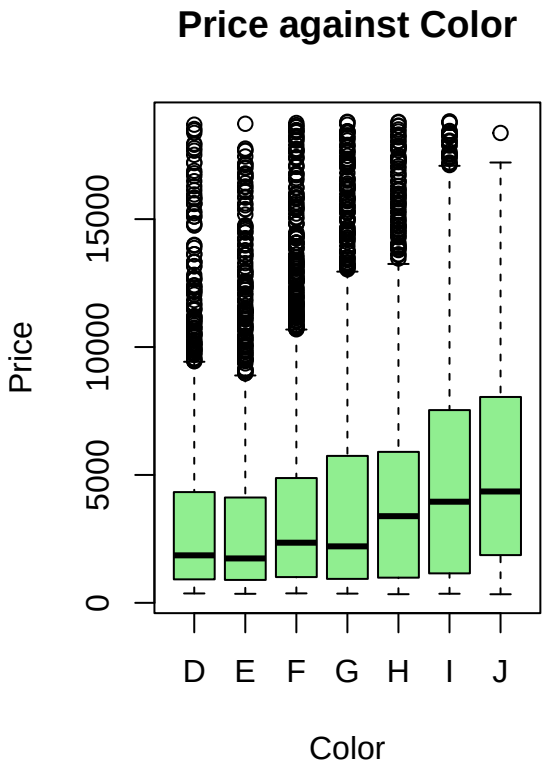


Looking at the variables' correlation matrix, we see that there's a positive correlation between the carat and price of a diamond (more carats, higher the price), and we also see that the greater the length of the diamond, the higher the carat AND price. Overall, there are definitely some correlations between all three variables.

```
# price and cut
par(mfrow = c(1, 2))
boxplot(price ~ cut, data.samp, main = "Price against Cut", xlab = "Cut",
    ylab = "Price", col = "lightblue")

# price and color
boxplot(price ~ color, data.samp, main = "Price against Color",
    xlab = "Color", ylab = "Price", col = "lightgreen")
```





Looking at the price against the cut, we see there's a fair correlation between the two. The premium cuts yield the maximum highest price, and with each tier under premium, the max price lowers. Looking at the price against color, we see that there's a pretty significant correlation between the two as the prices of the diamonds go up with a 'better quality' color (larger letter).

4.

```
multi.model <- lm(price ~ ., data.samp)

summary(multi.model)
```

```
##
## Call:
## lm(formula = price ~ ., data = data.samp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15995.9   -604.3   -174.8    379.2   9841.7
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.015e+03  9.896e+02   3.046 0.002323 **
## X             5.678e-03  7.688e-04   7.385 1.63e-13 ***
## carat         1.143e+04  1.168e+02  97.901  < 2e-16 ***
## cutGood       4.755e+02  7.339e+01   6.479 9.65e-11 ***
## cutIdeal      7.192e+02  7.266e+01   9.898  < 2e-16 ***
```

```
## cutPremium    7.073e+02  6.970e+01  10.147  < 2e-16 ***
## cutVery Good  5.742e+02  7.111e+01   8.074 7.50e-16 ***
## colorE       -1.873e+02  3.901e+01  -4.800 1.61e-06 ***
## colorF       -1.971e+02  3.971e+01  -4.965 6.99e-07 ***
## colorG       -4.349e+02  3.888e+01 -11.186  < 2e-16 ***
## colorH       -9.421e+02  4.130e+01 -22.813  < 2e-16 ***
## colorI       -1.437e+03  4.641e+01 -30.956  < 2e-16 ***
## colorJ       -2.341e+03  5.702e+01 -41.059  < 2e-16 ***
## clarityIF     5.110e+03  1.081e+02  47.288  < 2e-16 ***
## claritySI1    3.507e+03  9.078e+01  38.626  < 2e-16 ***
## claritySI2    2.575e+03  9.115e+01  28.254  < 2e-16 ***
## clarityVS1    4.482e+03  9.319e+01  48.096  < 2e-16 ***
## clarityVS2    4.134e+03  9.150e+01  45.175  < 2e-16 ***
## clarityVVS1   4.859e+03  9.966e+01  48.762  < 2e-16 ***
## clarityVVS2   4.871e+03  9.624e+01  50.613  < 2e-16 ***
## depth        -7.851e+01  1.183e+01  -6.639 3.32e-11 ***
## table        -2.293e+01  6.339e+00  -3.617 0.000299 ***
## x            -2.001e+03  2.062e+02  -9.703  < 2e-16 ***
## y             7.944e+02  2.077e+02   3.825 0.000132 ***
## z             2.273e+02  1.258e+02   1.806 0.070946 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1121 on 10763 degrees of freedom
## Multiple R-squared:  0.9201, Adjusted R-squared:  0.9199
## F-statistic:  5161 on 24 and 10763 DF,  p-value: < 2.2e-16
```

Looking at the multi linear regression model, we do see that y and z aren't statistically significant. In addition to this, we have a pretty accurate model with an adjusted R-Squared of 0.9206. This makes me believe that we can simply use a backwards step elimination process to achieve a pretty strong and accurate model made up of more statistically significant variables.

5. The data is pretty much what I expected, and it seems correlated in the way that I'd imagine in real life. I was incredibly surprised to see color playing a larger part in the price though, so that was interesting.

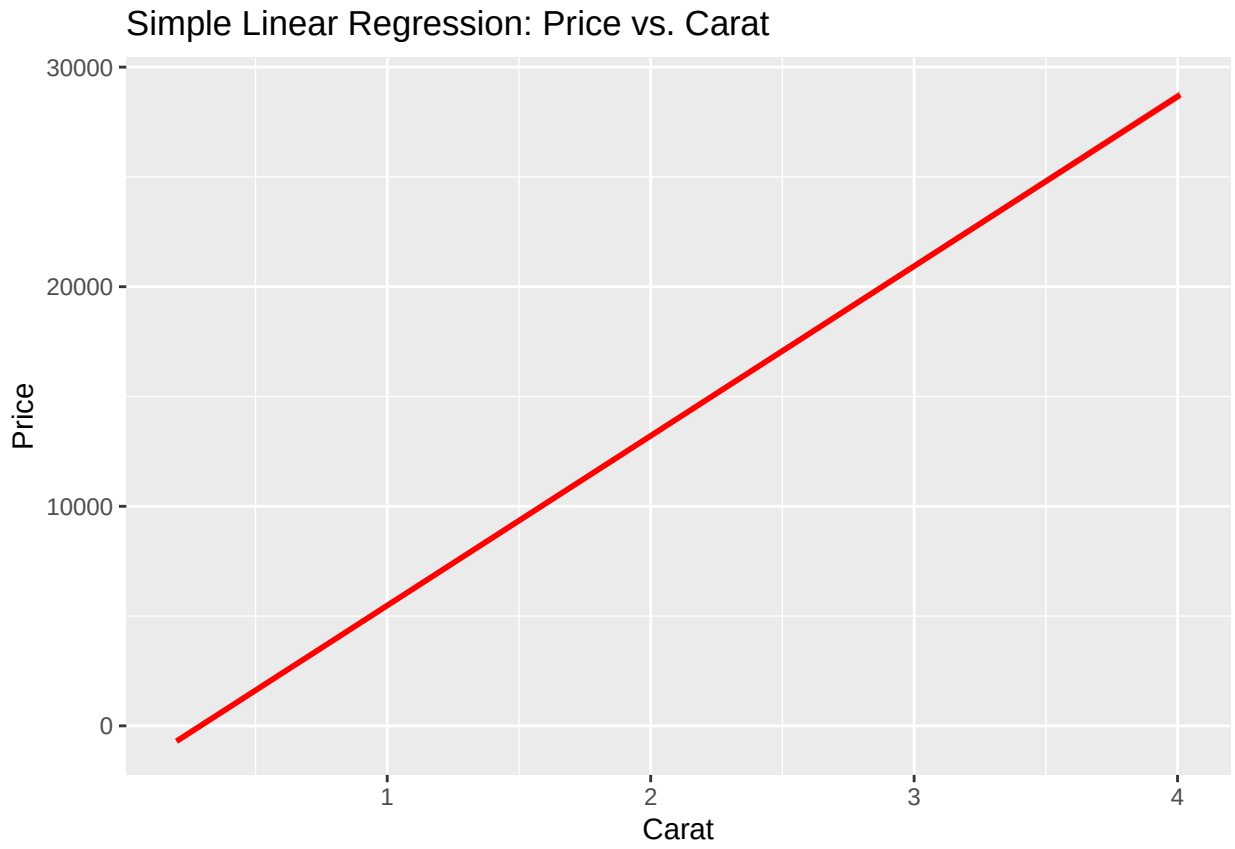
Part 2:

1.

```
model1 <- lm(price ~ carat, data.samp)

ggplot(data.samp, aes(x = carat, y = price)) + geom_smooth(method = "lm",
    color = "red") + labs(title = "Simple Linear Regression: Price vs. Carat",
    x = "Carat", y = "Price")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



2.

```
summary(model1)
```

```
##
## Call:
## lm(formula = price ~ carat, data = data.samp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13517.3   -803.3    -30.1    533.9  11887.4
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2246.81      29.61  -75.88   <2e-16 ***
## carat        7727.47      31.89  242.33   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1561 on 10786 degrees of freedom
## Multiple R-squared:  0.8448, Adjusted R-squared:  0.8448
## F-statistic: 5.873e+04 on 1 and 10786 DF,  p-value: < 2.2e-16
```

```r
confint(model1)
```

```
##                 2.5 %    97.5 %
## (Intercept) -2304.856 -2188.768
## carat        7664.965  7789.977
```

```r
predint <- predict(model1, interval = "prediction")
```

```
## Warning in predict.lm(model1, interval = "prediction"): predictions on current data refer to _future
```

```r
head(predint)
```

```
##         fit       lwr      upr
## 1 -392.2192 -3451.969 2667.531
## 2 2698.7691  -360.797 5758.335
## 3 4707.9115  1648.355 7767.468
## 4 1616.9232 -1442.684 4676.530
## 5 5712.4827  2652.899 8772.066
## 6 5557.9333  2498.355 8617.511
```

After printing out the summary statistics for this model, we can tell that from the coefficient's Pr(>|t|), the p-value is less than 0, which is well below the 0.05 significance level, indicating that we reject the null hypothesis. This means this variable is a statistically significant predictor of the diamond's price.

According to the Residual Standard Error, for every estimate, we'll be an average of $1540 off.

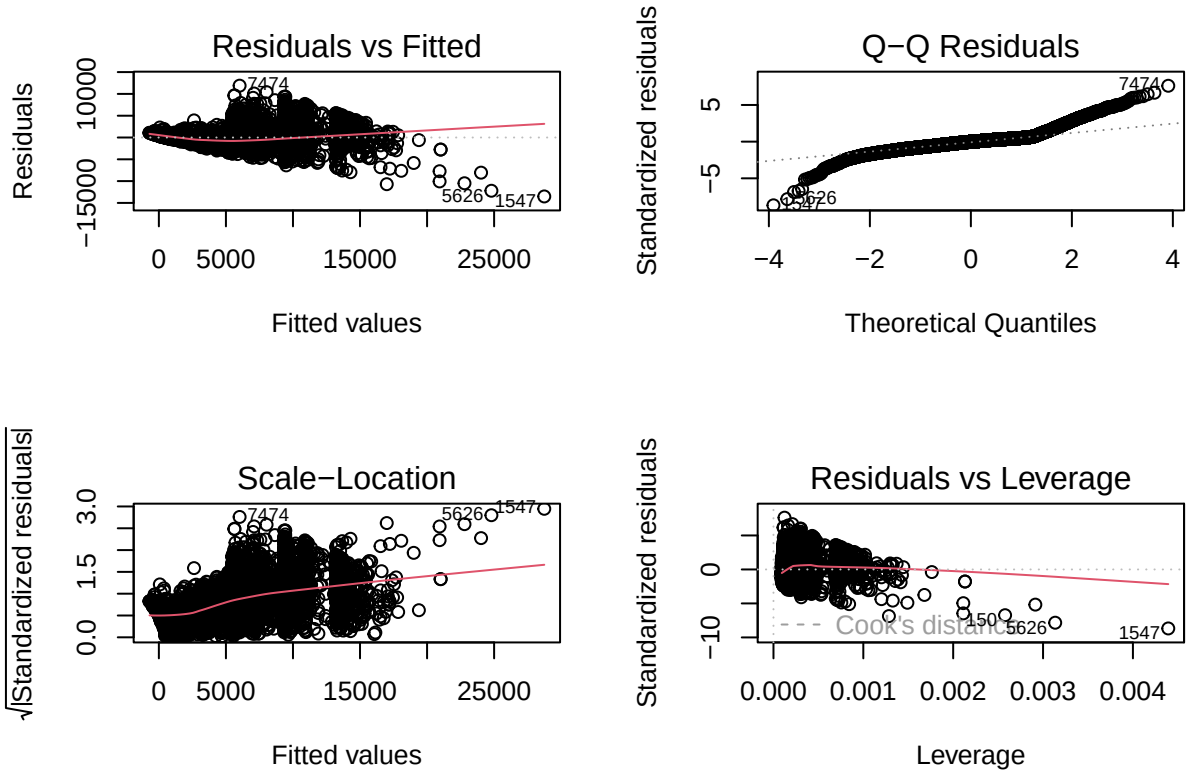
Looking at the F-Statistics, we have an incredibly large number, indicating that we reject the null hypothesis, suggesting at least one of the coefficients is statistically significant.

According to the adjust R-Squared, this model is already fairly solid. We sit at a value of 0.8477, which is incredibly close to the limit of 1, indicating we have a strong simple linear model that can estimate a diamond's price based on its carat.

The confidence interval of [7687.348,7811.316] shows that we're 95% confident that for every increase in a diamond's carat, the true average price of the diamond increases by an amount within that interval. The prediction interval indicates that for the first diamond of our set, the real price sits between 3021.4733 and 9059.298 dollars, with an estimated 6040.39 dollars from the model. However, I noticed that for some of the other diamonds, there are negative values, which indicates that some form of transformations are needed.

3.

```r
par(mfrow = c(2, 2))
plot(model1)
```

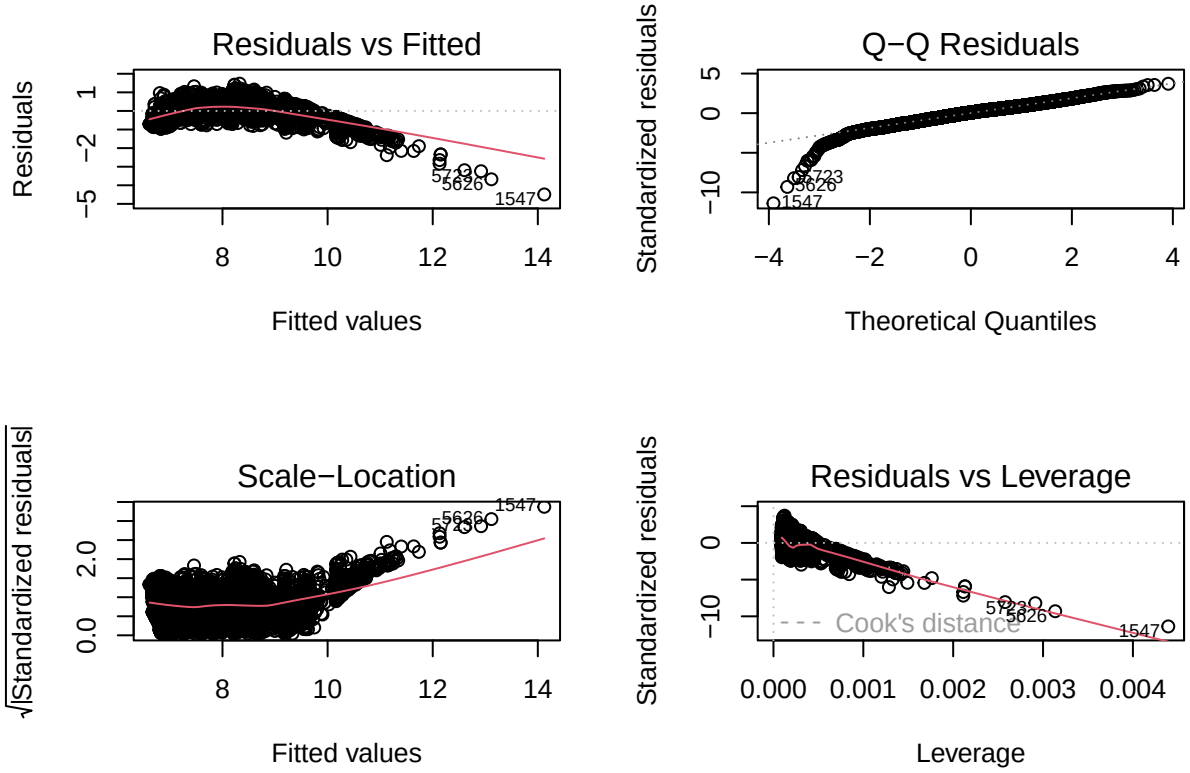



```
par(mfrow = c(1, 1))
```

We see the violation of equal variance in the residual vs fitted and a lack of normality in the Q-Q Residuals since it has an S shape, so we must log the model to make it not violate our assumptions, like so:

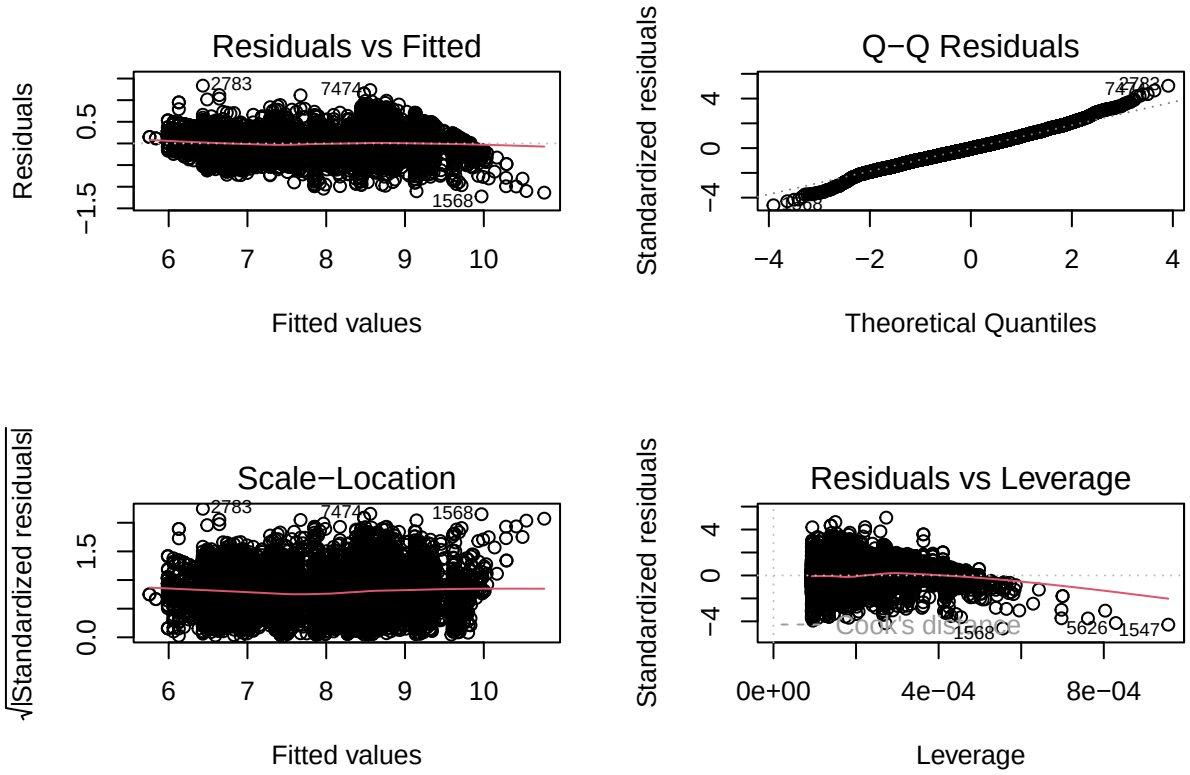
```
# attempting just y

y.model <- lm(log(price) ~ carat, data = data.samp)
par(mfrow = c(2, 2))
plot(y.model)
```




```
par(mfrow = c(1, 1))

# attempting both y and x
both.model <- lm(log(price) ~ log(carat), data = data.samp)
par(mfrow = c(2, 2))
plot(both.model)
```




```
par(mfrow = c(1, 1))
```

Now that the residuals and the fitted look significantly better (more randomized, thus the variance assumption is good), and the Q-Q Residuals looks less like a snake, meaning our normality assumption isn't violated.

4.

```
summary(both.model)
```

```
##
## Call:
## lm(formula = log(price) ~ log(carat), data = data.samp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.22459 -0.16870 -0.00987  0.16398  1.33495
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 8.446345   0.003073  2748.2   <2e-16 ***
## log(carat)  1.671419   0.004380   381.6   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.2652 on 10786 degrees of freedom
## Multiple R-squared:  0.931,  Adjusted R-squared:  0.931
## F-statistic: 1.456e+05 on 1 and 10786 DF,  p-value: < 2.2e-16
```

5. After going through all almost all of the variables and cutting the ones that don't increase the adjusted R^2...

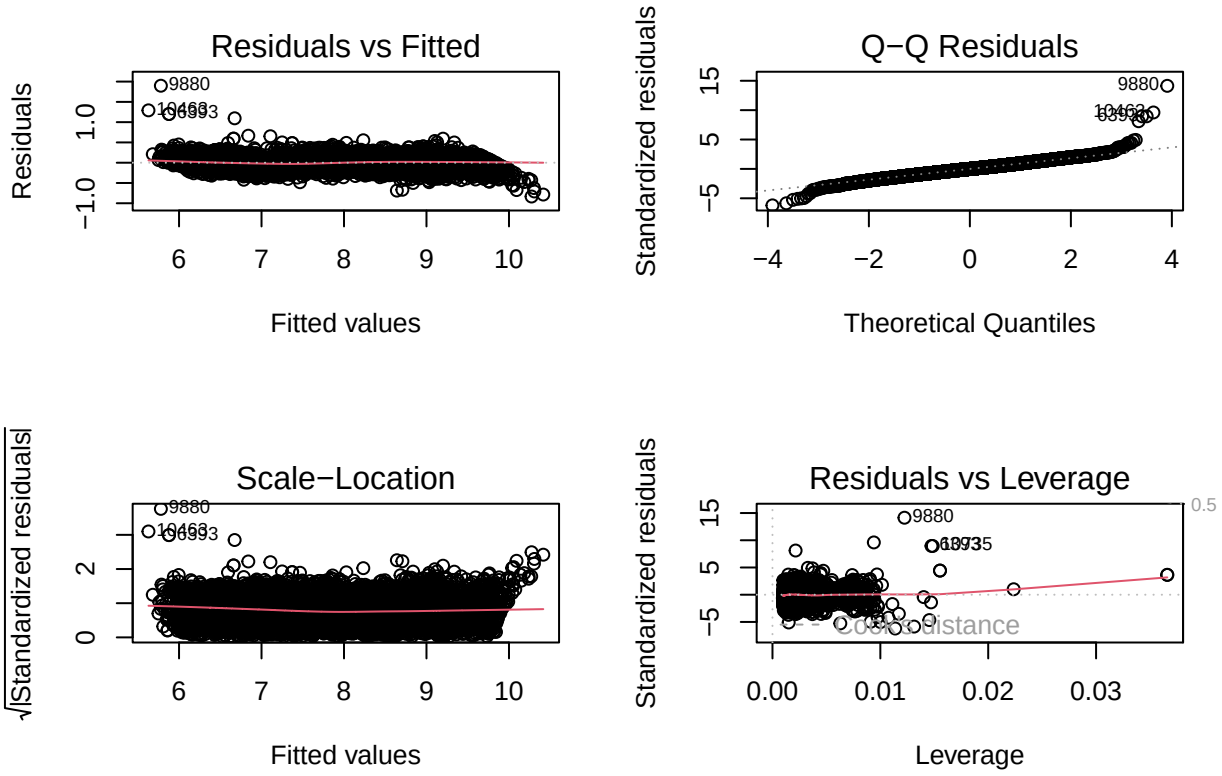
The final regression model is:

```
model6 <- lm(log(price) ~ log(carat) + x + color + cut + clarity +
    depth + table, data = data.samp)
summary(model6)
```

```
##
## Call:
## lm(formula = log(price) ~ log(carat) + x + color + cut + clarity +
##     depth + table, data = data.samp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.83369 -0.08365 -0.00076  0.08309  1.89801
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)   6.4402614  0.1432862  44.947  < 2e-16 ***
## log(carat)    1.5880024  0.0215008  73.858  < 2e-16 ***
## x             0.1553424  0.0112524  13.805  < 2e-16 ***
## colorE       -0.0561851  0.0046991 -11.957  < 2e-16 ***
## colorF       -0.0897149  0.0047826 -18.759  < 2e-16 ***
## colorG       -0.1532587  0.0046827 -32.729  < 2e-16 ***
## colorH       -0.2528936  0.0049733 -50.850  < 2e-16 ***
## colorI       -0.3748111  0.0055830 -67.135  < 2e-16 ***
## colorJ       -0.5186955  0.0068555 -75.661  < 2e-16 ***
## cutGood       0.0776784  0.0087120   8.916  < 2e-16 ***
## cutIdeal      0.1573838  0.0086641  18.165  < 2e-16 ***
## cutPremium    0.1328897  0.0083874  15.844  < 2e-16 ***
## cutVery Good  0.1160908  0.0083684  13.873  < 2e-16 ***
## clarityIF     1.0841867  0.0129523  83.706  < 2e-16 ***
## claritySI1    0.5712582  0.0108809  52.501  < 2e-16 ***
## claritySI2    0.4078625  0.0109364  37.294  < 2e-16 ***
## clarityVS1    0.7902771  0.0111672  70.768  < 2e-16 ***
## clarityVS2    0.7221761  0.0109657  65.858  < 2e-16 ***
## clarityVVS1   0.9979649  0.0119537  83.486  < 2e-16 ***
## clarityVVS2   0.9268427  0.0115371  80.336  < 2e-16 ***
## depth         0.0056240  0.0011488   4.896 9.93e-07 ***
## table         0.0015390  0.0007656   2.010   0.0444 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1351 on 10766 degrees of freedom
## Multiple R-squared:  0.9821, Adjusted R-squared:  0.9821
## F-statistic: 2.82e+04 on 21 and 10766 DF,  p-value: < 2.2e-16
```

However, I would like to check that it doesn't violate any of our assumptions:

```
par(mfrow = c(2, 2))
plot(model6)
```




```
par(mfrow = c(1, 1))
```

6. Since we're good, we can say this is our final personalized model! We have an adjusted R^2 of 0.9828, which means it's an incredibly strong model. In addition to this, apparently, according to the RSE, on average, for every estimation, we'll only be 0.1328 off. While this model is great, I would like to note that we haven't checked for multicollinearity. Seeing as we'll do it later in the project, we'll leave that section for that part.

Part 3

While I may use the following model from Part 2, curiosity makes me want to try the AIC method:

```
full.model <- lm(price ~ ., data.samp)

step(full.model, direction = "backward")
```

```
## Start:  AIC=151540.8
## price ~ X + carat + cut + color + clarity + depth + table + x +
```

```
##     y + z
##
##           Df  Sum of Sq        RSS    AIC
## <none>                  1.3537e+10 151541
## - z        1 4.1024e+06 1.3541e+10 151542
## - table    1 1.6459e+07 1.3554e+10 151552
## - y        1 1.8397e+07 1.3556e+10 151553
## - depth    1 5.5432e+07 1.3593e+10 151583
## - X        1 6.8603e+07 1.3606e+10 151593
## - x        1 1.1841e+08 1.3656e+10 151633
## - cut      4 1.6134e+08 1.3699e+10 151661
## - color    6 3.4654e+09 1.7003e+10 153988
## - clarity  7 7.2555e+09 2.0793e+10 156157
## - carat    1 1.2055e+10 2.5592e+10 158409

##
## Call:
## lm(formula = price ~ X + carat + cut + color + clarity + depth +
##     table + x + y + z, data = data.samp)
##
## Coefficients:
##  (Intercept)             X         carat       cutGood      cutIdeal
##    3.015e+03     5.677e-03     1.143e+04     4.755e+02     7.192e+02
##   cutPremium  cutVery Good        colorE        colorF        colorG
##    7.073e+02     5.742e+02    -1.873e+02    -1.971e+02    -4.349e+02
##       colorH        colorI        colorJ     clarityIF    claritySI1
##   -9.421e+02    -1.437e+03    -2.341e+03     5.110e+03     3.507e+03
##   claritySI2    clarityVS1    clarityVS2   clarityVVS1   clarityVVS2
##    2.575e+03     4.482e+03     4.134e+03     4.859e+03     4.871e+03
##        depth         table             x             y             z
##   -7.851e+01    -2.293e+01    -2.001e+03     7.944e+02     2.273e+02
```

```r
new.model <- lm(formula = price ~ X + carat + cut + color + clarity +
    depth + table + x, data = data.samp)

summary(new.model)
```

```
##
## Call:
## lm(formula = price ~ X + carat + cut + color + clarity + depth +
##     table + x, data = data.samp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16004.4   -600.6   -173.1    376.0   9878.3
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.506e+03  8.639e+02   2.901 0.003730 **
## X             5.530e-03  7.688e-04   7.193 6.74e-13 ***
## carat         1.146e+04  1.165e+02  98.426  < 2e-16 ***
## cutGood       5.223e+02  7.249e+01   7.204 6.24e-13 ***
## cutIdeal      7.588e+02  7.211e+01  10.523  < 2e-16 ***
```

```
## cutPremium    7.143e+02  6.974e+01  10.241  < 2e-16 ***
## cutVery Good  6.345e+02  6.972e+01   9.101  < 2e-16 ***
## colorE       -1.868e+02  3.905e+01  -4.784 1.74e-06 ***
## colorF       -1.967e+02  3.975e+01  -4.950 7.55e-07 ***
## colorG       -4.353e+02  3.891e+01 -11.188  < 2e-16 ***
## colorH       -9.440e+02  4.133e+01 -22.838  < 2e-16 ***
## colorI       -1.436e+03  4.645e+01 -30.909  < 2e-16 ***
## colorJ       -2.342e+03  5.706e+01 -41.051  < 2e-16 ***
## clarityIF     5.152e+03  1.077e+02  47.855  < 2e-16 ***
## claritySI1    3.538e+03  9.050e+01  39.096  < 2e-16 ***
## claritySI2    2.602e+03  9.096e+01  28.602  < 2e-16 ***
## clarityVS1    4.516e+03  9.288e+01  48.620  < 2e-16 ***
## clarityVS2    4.165e+03  9.121e+01  45.665  < 2e-16 ***
## clarityVVS1   4.894e+03  9.935e+01  49.260  < 2e-16 ***
## clarityVVS2   4.905e+03  9.592e+01  51.140  < 2e-16 ***
## depth        -6.859e+01  8.952e+00  -7.662 1.98e-14 ***
## table        -2.439e+01  6.334e+00  -3.850 0.000119 ***
## x            -1.085e+03  4.987e+01 -21.759  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1123 on 10765 degrees of freedom
## Multiple R-squared:  0.9199, Adjusted R-squared:  0.9197
## F-statistic:  5619 on 22 and 10765 DF,  p-value: < 2.2e-16
```

I'm noticing that the model using backward elimination closely resembles my model from before, but with the inclusion of X, and the removal of the log of carat. I'm also noticing that the adjusted R-Squared for this model is lower than the model I created, which is mildly strange. I'm going to remove X because it's simply the index of each diamond value (the rows), which shouldn't have any correlation.

2.

```
library(car)
```

```
## Warning: package 'car' was built under R version 4.4.3
```

```
## Loading required package: carData
```

```
## Warning: package 'carData' was built under R version 4.4.3
```

```
##
## Attaching package: 'car'
```

```
## The following object is masked from 'package:dplyr':
##
##     recode
```

```
vif_values <- vif(new.model)
vif_values
```

```
##              GVIF Df GVIF^(1/(2*Df))
## X        1.237157  1        1.112276
## carat   25.795749  1        5.078952
## cut      1.910859  4        1.084310
## color    1.204938  6        1.015657
## clarity  1.366911  7        1.022576
## depth    1.475132  1        1.214550
## table    1.730839  1        1.315613
## x       26.506653  1        5.148461
```

We definitely have some multicollinearity with carat and x, meaning we'll probably just have to remove x.

```
new.model <- lm(formula = price ~ carat + cut + color + clarity +
    depth + table, data = data.samp)
summary(new.model)
```

```
##
## Call:
## lm(formula = price ~ carat + cut + color + clarity + depth +
##     table, data = data.samp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12636.2   -680.3   -195.7    457.8   9576.2
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -4487.981    810.938  -5.534 3.20e-08 ***
## carat         8903.473     27.197 327.368  < 2e-16 ***
## cutGood        563.542     74.392   7.575 3.88e-14 ***
## cutIdeal       819.266     73.987  11.073  < 2e-16 ***
## cutPremium     773.837     71.555  10.815  < 2e-16 ***
## cutVery Good   701.506     71.471   9.815  < 2e-16 ***
## colorE        -197.437     40.135  -4.919 8.81e-07 ***
## colorF        -233.721     40.819  -5.726 1.06e-08 ***
## colorG        -465.652     39.973 -11.649  < 2e-16 ***
## colorH        -958.591     42.457 -22.578  < 2e-16 ***
## colorI       -1391.861     47.699 -29.180  < 2e-16 ***
## colorJ       -2273.312     58.571 -38.813  < 2e-16 ***
## clarityIF     5240.504    110.589  47.387  < 2e-16 ***
## claritySI1    3435.756     92.914  36.978  < 2e-16 ***
## claritySI2    2495.673     93.387  26.724  < 2e-16 ***
## clarityVS1    4467.646     95.438  46.812  < 2e-16 ***
## clarityVS2    4104.635     93.706  43.803  < 2e-16 ***
## clarityVVS1   4960.388    102.080  48.593  < 2e-16 ***
## clarityVVS2   4925.821     98.580  49.968  < 2e-16 ***
## depth          -20.907      8.895  -2.350 0.018778 *
## table          -23.973      6.507  -3.684 0.000231 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1154 on 10767 degrees of freedom
## Multiple R-squared:  0.9153, Adjusted R-squared:  0.9152
## F-statistic:  5821 on 20 and 10767 DF,  p-value: < 2.2e-16
```

```r
vif_values <- vif(new.model)
vif_values
```

```
##             GVIF Df GVIF^(1/(2*Df))
## carat   1.331050  1        1.153711
## cut     1.899351  4        1.083492
## color   1.186352  6        1.014342
## clarity 1.311132  7        1.019538
## depth   1.378811  1        1.174228
## table   1.728964  1        1.314901
```

3.

```r
new_diamond <- data.frame(carat = 1, cut = "Ideal", color = "G",
    clarity = "VS2", depth = 61.5, table = 57)

predict(new.model, new_diamond, interval = "confidence")
```

```
##        fit      lwr      upr
## 1 6221.494 6149.464 6293.524
```

```r
predict(new.model, new_diamond, interval = "prediction")
```

```
##        fit      lwr      upr
## 1 6221.494 3958.634 8484.354
```

We see a prediction of 6193.422, with a Confidence Interval of (6123.466,6263,379) and Prediction interval (3957.32,8429.526).

4. Overall, we found a pretty strong model with an Adjusted R-Squared of 0.9206! It includes the carat, cut, color, clarity, depth and table of a diamond, and can fairly predict the price of a diamond given those variables! While the customized and checked model that we made in part 2, which includes the transformed price and carat variables, had a better R-Squared, it was still incredibly interesting to see that the backward step elimination had a fairly similar variable makeup. The only difference between the two is that there was no log transformation, and we also omit x from the backward step elimination model because of multicollinearity. Using the backward step elimination model, we were able to get a pretty accurate prediction for a diamond with a carat of 1, an ideal cut, the color "G", a clarity of VS2, a depth of 61.5 and a table of 57 to have a price of around 6193.422.